Structural verification of index invariants in a running database

Andrey Borodin

Аннотация

A defect in an index access method does not announce itself. The index does not fail; it begins to answer incompletely, and a query whose answer exists in the table does not find it. The application keeps working, and the discrepancy surfaces at an arbitrary later time, when the cause can no longer be reconstructed. Three defects of exactly this kind are recounted here from our own work on PostgreSQL: an inconsistent comparator in sorted index construction, the premature return of deleted pages to circulation, and prepared transactions overlooked when building an index concurrently. None of them is detectable by functional testing, because every individual operation returns a correct result; what fails is a property of the structure as a whole.

We report the structural invariants of the generalized search tree and the generalized inverted index, stated in terms of the operator class interface so that they hold for any key type, and a verification procedure for them that is usable in production: on a live system, without exclusive access to the index, with memory bounded by the width of one tree level, and on a physical replica so that the load falls somewhere other than on the system serving users. We describe the two techniques that make this possible — a level-order traversal that carries parent keys downward through intermediate storage, and sibling-link verification with lock coupling in a single direction — and the normalization of index tuples without which comparing a tuple against a freshly built one compares representations rather than values.

We also report that the checker itself needed checking, and that the shape of its defects follows from its design contract. Since the first version, amcheck has forbidden false positives absolutely, conceding in exchange that it cannot prove the absence of corruption. That trade is right, but it spends the whole error budget on silence: where a check cannot be certain it declines, and a declined check produces no signal. Three of the four defects found in the first year were guards written so that the check behind them almost never ran — success reported on indexes never examined. Silent incompleteness is the property the tool exists to detect, and it is the property it exhibited. The remedy is not to loosen the contract but to measure coverage directly.

1 Introduction

An index is a redundant structure: everything it contains is derivable from the table it indexes. That redundancy is what makes it fast, and it is also what makes its failures quiet. If the index and the table disagree, the database does not stop; it answers a query with a subset of the correct answer, and no layer of the system is in a position to notice.

This is not a hypothetical failure mode. Three defects from our own work on PostgreSQL's generalized search tree fall squarely into it. A comparator used in sorted construction was inconsistent for keys that mapped to equal values, producing a tree whose ordering assumptions did not hold. Deleted pages could be returned to circulation before every scan that might reach them had finished, so a scan could read a page holding unrelated data. And building an index concurrently did not account for prepared transactions, so rows committed through two-phase commit could be silently absent from it. In each case every individual operation returned a correct result. What was violated was a property of the structure as a whole, and no test of individual operations can observe such a property.

The conclusion we draw is that a class of correctness properties in a storage engine needs a different discipline from testing behaviour: state the invariants of the structure, and check them against the actual bytes on disk. That conclusion is not novel in itself — it is the premise of amcheck for the B-tree [4]. What this paper contributes is its extension to an extensible index framework, where the key type is supplied

by the user and the checker cannot know what a key means, and its extension to production conditions, where the check must run without stopping the workload.

Contributions.

1. Structural invariants for the generalized search tree and the generalized inverted index, expressed entirely through the operator class interface and therefore valid for any key type (Section 2).
2. A level-order traversal that verifies parent key consistency with memory bounded by the width of one tree level rather than by the size of the index (Section 3).
3. Sibling-link verification with lock coupling, and the argument that makes coupling admissible in a checker (Section 4).
4. Normalization of index tuples, without which comparison against a regenerated tuple compares representations rather than values (Section 5).
5. An account of running the check on a physical replica, and of the defects found in the checker itself after release (Sections 6 and 8).

2 Why the operator class interface is the right vocabulary

A generalized search tree [5] is a balanced tree whose node occupies a page and whose entries pair a key with a downlink, the key covering every key in the subtree below. The key type and its operations belong to an operator class supplied outside the framework: `consistent`, `union`, `penalty`, `picksplit` and the rest. The framework does not know whether a key is a rectangle, a signature or a range, and a verification tool built into the framework cannot know either.

This is a constraint, and it is also the design principle. If the invariants are stated using only the operator class interface, the checker inherits the extensibility of the framework: it works for a key type written after the checker was. If they are stated in terms of any particular key semantics, it does not. Every invariant below is therefore expressed through `consistent` alone, which is the operation the search itself uses.

1. *Parent key consistency.* For every entry of an internal page, the key of that entry covers the keys of all entries on the page it points to: $\texttt{consistent}(k_{\text{parent}}, k_{\text{child}})$ holds for every child. A violation means a search will not descend into a subtree that contains an answer — silently incomplete results, the failure mode of Section 1.
2. *Balance.* All leaf pages lie at the same depth, and an internal page points either only to internal pages or only to leaf pages.
3. *Completeness of the right-link chain.* A page's right link points to a page of the same level, and following the chain visits every page of the level exactly once.
4. *Sibling agreement.* The right link of a left neighbour points to a page whose left link points back to it.
5. *Unreachability of deleted pages.* No entry of any internal page points to a page marked deleted.

Invariant 3 has a consequence for construction that is worth stating explicitly, because it runs the other way from how such things are usually decided. Bulk loading a tree bottom-up could leave right links unset, since no search needs them: a search descends. Leaving them unset would force the checker to distinguish indexes by how they were built, and an invariant that holds only for some instances of a structure is not an invariant. The construction was therefore made to set them. Verifiability is here a design input rather than an afterthought.

For the inverted index the object is not a tree but a graph: an entry tree whose leaves hold either inline posting lists or the roots of posting trees. The invariants restate accordingly — key consistency along paths of the graph, at least one downlink per internal page, uniformity of the level an internal page points to, and ordering of entries within a page.

3 Verifying parent keys without holding the index

The direct way to check invariant 1 is to descend from the root, carrying each parent key to its children. This is correct and unusable in production for two reasons: the descent reads pages in an order unrelated to their placement, and it holds state proportional to the width of the level being descended into, which for a large index means holding most of the index.

The procedure used instead traverses in level order and passes parent keys downward through intermediate storage. Memory is then bounded by the width of one level rather than by the size of the index, and the pages of each level are read in increasing page number — the same argument as for maintenance, and for the same reason.

The check runs under share locks and does not block readers or writers. It therefore observes a moving structure, and its invariants must be ones that hold of every intermediate state a concurrent modification passes through, not merely of quiescent states. This is a real restriction on what can be checked, and it is the reason invariant 1 is stated as coverage rather than as equality: a concurrent insertion may have already widened a parent key without yet having inserted the entry that motivated it, and coverage survives that window where equality would not.

4 Sibling links and lock coupling

Invariant 4 cannot be checked while holding a lock on one page: the check needs to see a left neighbour and its right neighbour at once. That means lock coupling — holding one page's lock while acquiring the next — which integrity tools had previously avoided, on the reasonable grounds that it complicates reasoning about deadlock.

The argument for admitting it here has two halves. Formally, the locks are taken strictly in one direction, left to right, so no cycle of waits can form with any other party that also respects that order; and they are share locks, so readers are unaffected. Practically, sibling verification detects a substantial share of the corruption actually encountered in the field at very low cost, which is what justifies the additional complexity. We state both halves because the first alone would not have been enough to change the practice, and the second alone would not have been enough to make it safe.

Implemented in `39132b784ae`, released in PostgreSQL 13; discussed in [2].

5 Normalizing tuples before comparison

The principal way to check that an index agrees with its table is to take an index tuple and compare it against a tuple built afresh from the table value. Byte comparison does not work. The same value of a variable-length type may be represented in several ways — compressed or not, with headers of different lengths — and which representation is present depends on the history of operations that produced it, not on the value. A checker comparing bytes reports differences that are not differences.

The fix is to normalize both tuples to a canonical representation before comparing: `b1fe8efdf17` for uncompressed variable-length data, and, by Michael Zhilin, `ab65dfb0fb2` for the differing header sizes of short variable-length data.

The sequel belongs in this paper rather than in a footnote. The normalization itself then turned out to use the wrong accessor for the size of a variable-length datum (`da1eff08a5b`), which is to say that the code written to remove a source of false positives introduced one. We return to this in Section 8.

6 Running the check on a replica

Verification reads the whole index, and a system serving users is the wrong place to do that. The practical requirement is therefore that the check run on a physical replica, which shifts the load off the primary and makes it affordable to run often. Two things had to be corrected for that to work.

Relations that are not fully logged have no meaningful content on a replica, and checking them produces reports that mean nothing; they are skipped (`6754fe65a4c`). And a check that needs a consistent view of the data must obtain one under recovery conditions, where the usual mechanism does not apply unchanged (`1f28982e409`).

There is a subtler dependency. On a replica the structure is the product of replaying the write-ahead log, so the correctness of the check is contingent on the correctness of that replay. Without the corresponding fixes, the check reports corruption that exists only in the replica's reconstruction of the index. This is worth stating plainly: verification on a replica checks the primary's index and the replay path at once, and does not by itself tell you which of them is at fault.

7 Making corruption a machine-readable event

A report of corruption is only useful to an operator if a program can act on it. Distinguishing corruption by the text of an error message is not something one can build automation on. Corruption reports were therefore given error codes of their own, in work by Melanie Plageman (`8ec97e78a77`), so that tooling can recognize the condition and, for instance, take a node out of service without a human reading a log.

This is not our change, and we include it because the discipline this paper argues for is incomplete without it: an invariant checked but reported only in prose is checked for a human, and the case for continuous verification rests on a machine being able to act on the answer.

8 The checker needed checking

Verification of the inverted index shipped in PostgreSQL 18 — `14ffaece0fb`, written with Grigory Kryachko and Heikki Linnakangas, with the common routines factored out separately in `d70b17636dd`. Since then four defects have been fixed in it: posting tree checks (`0cf205e122a`), the parent key check (`cdd1a431f21`), the ordering of entry-tree items (`0b54b392334`), and a memory leak in the parent key consistency pass reported from outside [6]. Support for the generalized search tree is partial and still in review [3], seven years after the first attempt at it [1].

We report this deliberately, because the shape of these defects is not accidental. It is the shape the design asks for.

The contract. From the first version of amcheck, Geoghegan fixed one property as inviolable: the tool must never report corruption in a sound structure. In the original design discussion, on a race that could produce a spurious report: "I'm pretty determined that false positives like that need to be impossible (or else it's a bug)" [4]. The rule is stated in the code — *in general, false positives are disallowed* — and in the user documentation, which also states the corresponding concession: amcheck "can only prove the presence of corruption; it cannot prove its absence" [7].

The two sentences are one decision. The error budget of the tool is spent entirely on one side. Where a check cannot be certain — because the structure is being modified concurrently, because a page was deleted mid-traversal, because a parent key was widened before the entry that motivated it arrived — the prescribed response is to decline to check, not to report. Every such decision is locally correct and locally invisible.

The consequence. Three of our four defects lie in exactly that direction. The parent key check (`cdd1a431f21`) guarded its work with a test on the page's right link, written as a test for the link

being zero; an absent right link is recorded as the largest representable block number, not zero. The guard was therefore almost always false, the parent key was left unset, and the comparison never ran. The ordering check skipped comparisons across attributes of a multi-column index (`0b54b392334`). The posting tree check invalidated parent keys for more pages than necessary and so declined to use them (`0cf205e122a`). In each case the tool reported no corruption without having looked.

We do not think the contract is wrong. A false positive sends an operator hunting for damage that does not exist, and after that happens twice the tool is switched off, at which point its detection rate is zero however good it is; whereas a missed defect leaves the operator no worse off than before. Given that the tool cannot prove absence in any case, spending the entire budget on the safe side is the correct trade.

What the contract does not do is make its own cost observable. A gap in coverage produces no signal at all: the check runs, reports success, and nobody investigates a clean bill of health. This is the failure mode of Section 1 reappearing one level up — the index answers a query with a subset of the correct answer; the checker answers "is this index sound" with a subset of the checks it claims to perform — and it is invisible to functional testing for the same reason. A test that runs the checker on a good index and expects silence passes whether or not any check executed.

The remedy that follows is not to relax the contract but to measure coverage directly: for each invariant, corrupt a structure so that precisely that invariant is violated, and confirm that precisely that check fires. This is what Section 9 sets out to do, and it is the instrument that would have caught `cdd1a431f21` on the day it was written. Absent such a matrix, the gaps were found the way they were in fact found — by readers. The posting tree issues and the memory leak in the parent key pass were reported from outside by Arseniy Mukhin [6]; the normalization defect (`da1eff08a5b`), a variable-length size accessor used before the header size was known, by Andres Freund.

Where that leaves the tool. A verification tool is subject to the argument it exists to make. Its coverage is a structural property that no functional test of its callers can observe, and the evidence for it is the same kind of evidence it demands of everything else: deployment across many key types and workloads, over time, plus readers. Seven years from the first patch [1] to partial support still in review is not obviously too slow for something held to that standard.

9 Evaluation

9.1 Measuring coverage by deliberate corruption

Section 8 argued that the contract forbidding false positives spends the whole error budget on silence, and that a declined check produces no signal. The instrument that follows is a corruption matrix: violate one invariant at a time and require that the corresponding check fire.

The index is a generalized inverted index over 200 000 rows of integer arrays. Page checksums are disabled deliberately — otherwise the corruption is caught by the checksum layer rather than by an invariant check, whereas a defect originating inside the database would leave the checksum consistent. Before each case the index file is restored from a pristine copy, exactly one field is altered at the byte level, and `gin_index_check()` is run.

One firm gap. The right link of an entry-tree leaf was replaced by a non-existent block number and the checker stayed silent. This is not the weak claim that the page goes unread: on *the same page*, corrupting tuple order and corrupting `pd_lower` are both caught. The page is read, its contents are parsed, and the right link is not examined. Invariant 3 of Section 2 is unverified for this index type.

Two claims the controls withdrew. We record these because they are the method working, not the method failing.

Таблица 1: Coverage matrix. “Visited” records whether a control corruption on the same page fires, proving the checker reads it.

invariant violated	page	visited	detected
tuple order within an entry page	30	—	yes
`pd_lower` consistency (control)	30	—	yes
`maxoff` against `pd_lower`	2	—	yes
right-link chain completeness	30	yes	**no**
deleted page unreachable	3	no	inconclusive
parent key consistency	438	yes	unreliable

Marking a posting tree page deleted while a downlink to it survives produces no complaint — but neither does the control corruption on that same page, so we have no evidence the page is examined at all, and silence cannot be attributed to a missing check. Reported as inconclusive rather than as a gap.

For parent key consistency, the first attempt raised the key of the first tuple on an internal page; the order check fired, so two invariants had been violated at once and the detection could not be attributed. The second attempt used the last tuple to preserve order, and the checker was silent — which looked like a second gap. The third attempt, lowering the key so as to break coverage rather than widen it, revealed that the “last” tuple carried key 93 where its predecessor carried 19 937. The offsets were not addressing what we assumed, so the second attempt had not corrupted a parent key either, and the row is unreliable rather than a gap.

What this costs the method. To assert that an invariant is unchecked, silence is not enough. Two things must be shown: that the page is visited, which a control corruption on the same page establishes; and that what was corrupted is what was named, which requires parsing the on-page format rather than computing an offset from an assumption. Only the right-link row here satisfies both. That is one solid finding out of six attempts, and the discipline that reduced the other five is the same discipline this paper recommends applying to indexes.

9.2 Cost of the check

Two claims of Section 3 are measurable directly: that level-order traversal reads each page about once, and that memory is bounded by the width of one level rather than by the size of the index.

The first holds exactly. The ratio of pages read to pages in the index is **1.00** in every configuration measured — 1 134 of 1 135, 2 848 of 2 849, 10 770 of 10 771, the difference being the metapage. The quantity is deterministic and independent of cache size and warming.

Elapsed time follows from that: 0.5, 1.3 and 4.9 seconds for the three sizes on a cold cache, and 0.0, 0.0 and 0.1 seconds warm. The check is I/O bound and costs one sequential pass; on a warm index it is close to free. This is what makes the practical argument of Section 6 work — verification cheap enough to run continuously rather than during an incident.

9.3 A claim that one version would have refuted

The memory claim produced the most instructive result in this paper, and it is not the one we set out to obtain.

Measured on the current release, peak resident memory of the checking backend grew linearly with the index: 25.4, 33.8 and 74.6 MB as the index grew by a factor of 9.5. The buffer cache was 32 MB, so touched shared buffers cannot account for it. The reading was plain: memory is *not* bounded by level width, and Section 3 is wrong.

It is not wrong. A pair of builds bracketing a single commit — `1f8ab91c11e`, “amcheck: Fix memory leak with `gin_index_check()`” [6] — separates the algorithm from the build.

Таблица 2: Peak resident memory of the checking backend, 32 MB buffer cache

index pages	before the fix	after the fix
1 135	20.1 MB	14.4 MB
2 849	28.5 MB	14.3 MB
10 771	69.8 MB	**20.0 MB**

After the fix, memory is essentially flat across a ninefold growth of the index, which is the claim of Section 3. Before it, memory grows with the index, which is the leak: the commit message records that a tuple pointer was overwritten on every iteration, “leaking memory for every tuple processed on entry tree pages, the larger the worse”. Our measurement reproduces that independently and puts a number on it — 70 MB against 20 MB at five million rows.

The methodological point is worth more than the numbers. A measurement on a single version would have refuted a true claim, because that version contained a defect. No amount of care within one build separates “this is how the algorithm behaves” from “this is how this build behaves”; only a pair differing by one commit does. That is the same argument this paper makes about checking indexes, turned on the act of measuring: the silence of a check proves nothing, and neither does a single reading.

10 Limitations

Verification of the generalized search tree is partial: what is committed covers the inverted index, and the tree is still in review. The check runs concurrently and therefore verifies only invariants that hold of every intermediate state, so properties true merely of quiescent structures are outside its reach. Coverage of key types rests on the operator class interface being used as intended; an operator class whose `consistent` is itself wrong will be found consistent with itself. And on a replica the check cannot separate a defect in the index from a defect in log replay.

11 Conclusion

The failures that matter most in an index access method are the ones that do not look like failures. Every operation succeeds, the structure quietly stops agreeing with the table, and nothing in the ordinary path of the system is in a position to notice. Testing behaviour cannot reach this class, because the behaviour is correct at every step.

What reaches it is stating the invariants of the structure and checking the bytes against them. For an extensible framework this can be done without knowing what a key means, using only the interface the search itself uses, which is what makes the checker apply to key types written after it. Doing it on a running system constrains which invariants are available — only those that hold of every intermediate state — and doing it on a replica makes it affordable enough to run continuously rather than during incidents.

The last finding is about the instrument. An integrity checker is built under a contract that forbids false positives absolutely and concedes that absence of corruption cannot be proved. That trade is right: a tool that cries wolf is switched off and then detects nothing at all. But it spends the entire error budget on silence, and silence has no signal. Three of the four defects found in the checker’s first year were guards written so that the check behind them almost never ran, each locally defensible under the rule “do not report unless certain”, and together amounting to success reported on indexes never examined. The property the tool exists to detect is silent incompleteness, and silent incompleteness is what it exhibited. The remedy is not to loosen the contract but to measure coverage: violate each invariant deliberately and require that its check fire. A checker that is never wrong when it speaks still has to be made to speak.

Acknowledgements

The premise of this work is not ours. Peter Geoghegan built the invariant checker for the B-tree and established, from its first version, the rule that governs everything here: a report of corruption must never be false. He also committed the sibling-link and replica work of Sections 4 and 6, which was written jointly with him.

Verification of the inverted index was written with Grigory Kryachko and Heikki Linnakangas and committed by Tomas Vondra, who also committed the three corrections discussed in Section 8. Those corrections were found and written by Arseniy Mukhin, who has done more than anyone to establish what that checker actually checks; the memory leak of Section 9 was reported by him and fixed by Kirill Reshke, with review by Ewan Young, and committed by Michael Paquier.

Alexander Korotkov committed the normalization work and reviewed most of it; the handling of differing header sizes is Michael Zhilin's, and the defect in our own normalization was reported by Andres Freund. Álvaro Herrera committed the recovery fix, which is Mihail Nikalayeu's work. Alexander Lakhin and Jian He reviewed the normalization series. The error codes that make corruption machine-readable are Melanie Plageman's, reviewed by Robert Haas.

Support for the generalized search tree remains in review, and we thank those carrying it there: Arseniy Mukhin, Kirill Reshke, Japin Li, Roman Khapov, Miłosz Bieniek, Paul A. Jungwirth and Nikolay Samokhvalov. José Villanova, Aleksander Alekseev and Nikolay Samokhvalov reviewed the module refactoring.

Errors that remain are the author's.

Список литературы

[1] Andrey Borodin et al. amcheck verification for GiST. Discussion on the pgsql-hackers mailing list, 2018.

[2] Andrey Borodin et al. Amcheck: do rightlink verification with lock coupling. Discussion on the pgsql-hackers mailing list, 2019.

[3] Andrey Borodin et al. amcheck: support for GiST. Discussion on the pgsql-hackers mailing list, 2025.

[4] Peter Geoghegan et al. amcheck (B-tree integrity checking tool). Discussion on the pgsql-hackers mailing list, 2016.

[5] Joseph M Hellerstein, Jeffrey F Naughton, and Avi Pfeffer. *Generalized search trees for database systems*. University of Wisconsin-Madison, 1995.

[6] Arseniy Mukhin et al. GIN amcheck leaks memory in gin_check_parent_keys_consistency. Discussion on the pgsql-hackers mailing list, 2026.

[7] PostgreSQL Global Development Group. Postgresql documentation. chapter 65: Gist indexes. `https://www.postgresql.org/docs/current/gist.html`, 2026.